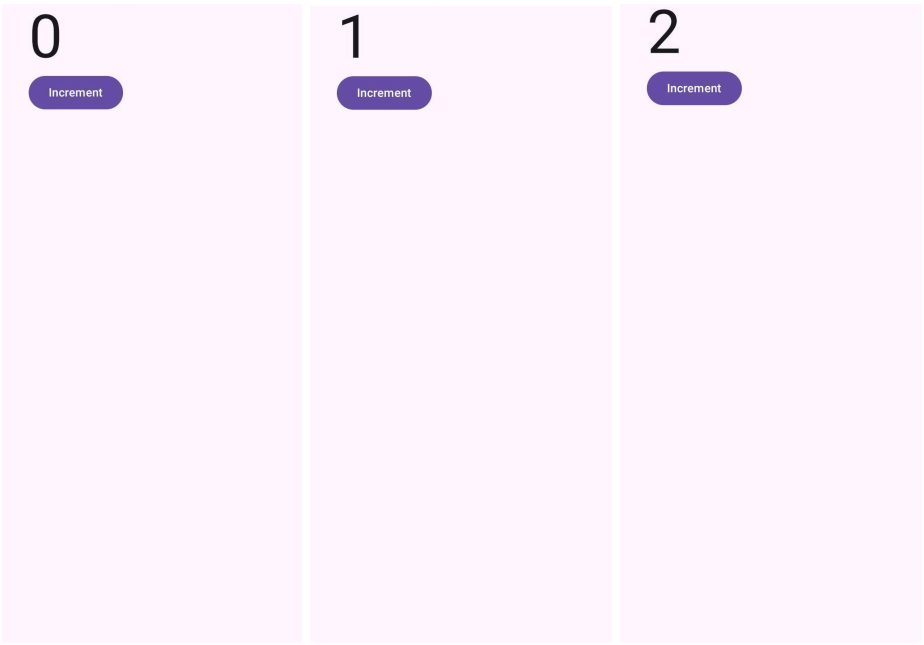
Tugas Modul Praktikum 4
State Hoisting, Perubahan Konfigurasi, dan Side-effect

Nama : Dennisa Syafa
NIM : 245150400111040
Matkul : Pengembangan Aplikasi Mobile - Kelas D

State Hoisting

Aktivitas Percobaan:

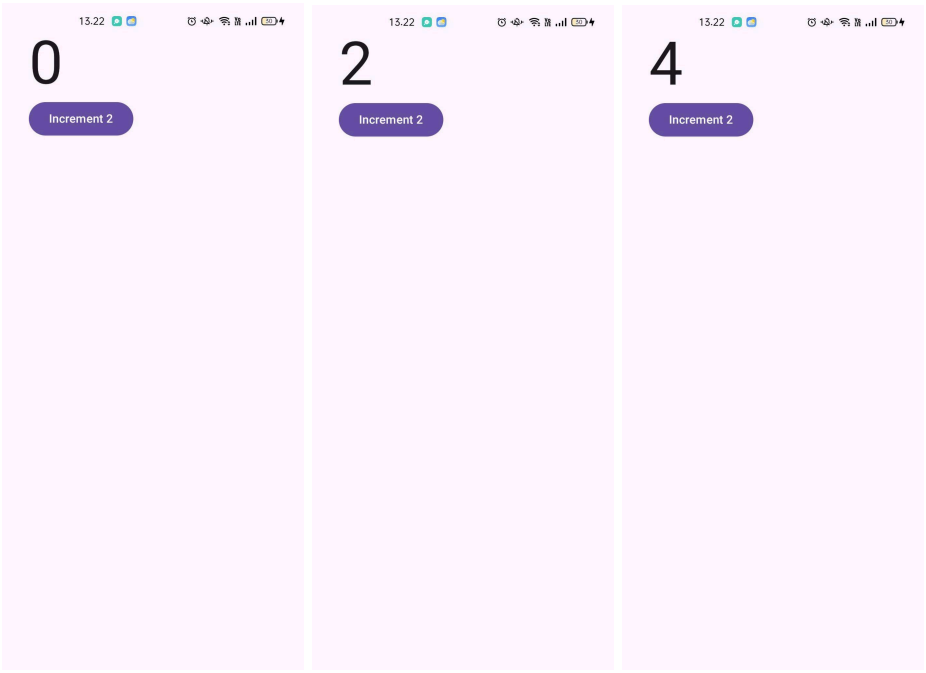
1. Hasil Program



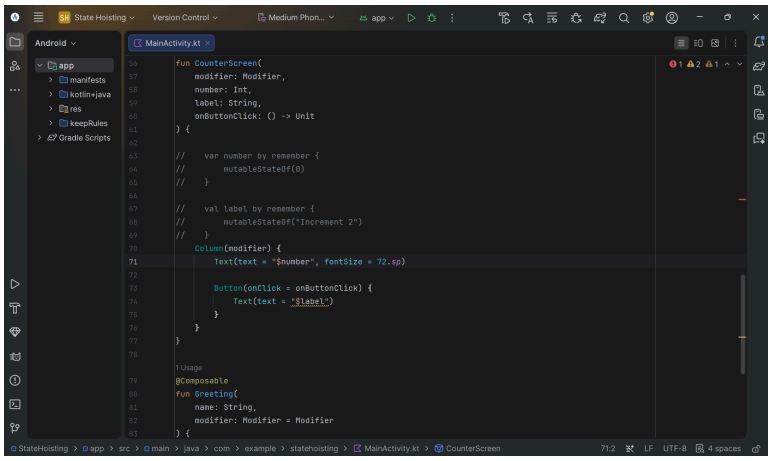
Jawaban:

a. Setiap kali tombol Increment diklik, nilai angka yang ditampilkan pada layar bertambah satu. Misalnya dari 0 menjadi 1, kemudian 2, 3, dan seterusnya.
b. Hal tersebut terjadi karena terdapat event onClick pada Button yang menjalankan number++. Perubahan nilai number sebagai state akan menyebabkan Compose melakukan recomposition, sehingga Text menampilkan nilai terbaru.
c. Variabel state yang berperan adalah number, yang dideklarasikan menggunakan mutableStateOf(0) dan diingat menggunakan remember. Variabel number digunakan oleh composable Text untuk menampilkan angka yang berubah.
d. Variabel state yang berperan adalah label, yang dideklarasikan menggunakan mutableStateOf("Increment") dan digunakan sebagai nilai pada Composable Text di dalam Button.

2. Perubahan kode
   Jawaban:
   a. Perubahan dilakukan pada fungsi CounterScreen(). Operasi number++ pada onClick diubah menjadi number += 2, sehingga nilai bertambah dua setiap kali tombol diklik. Selain itu, nilai label diubah dari "Increment" menjadi "Increment 2".
   b. Sebelum: Sesudah:



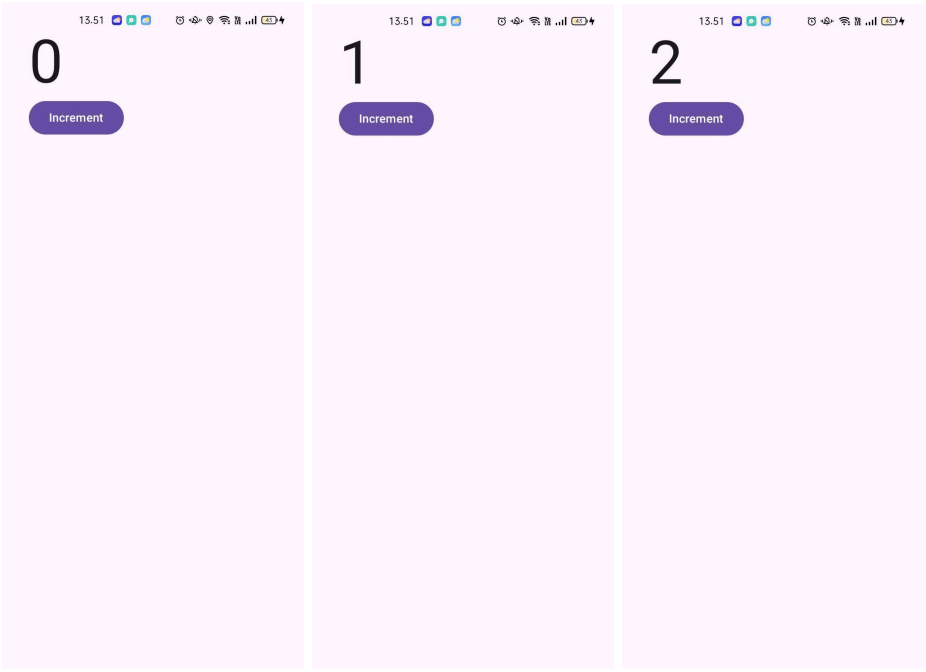
3. Perubahan kode program:



Pada tahap ini dilakukan state hoisting dan event hoisting dengan memindahkan pengelolaan state number dari CounterScreen() ke dalam onCreate(). CounterScreen() kemudian menerima number, label, dan onButtonClick sebagai parameter.

CounterScreen() tidak lagi mengelola state dan event secara langsung, tetapi hanya menampilkan data dan menjalankan event yang diberikan oleh pemanggil.

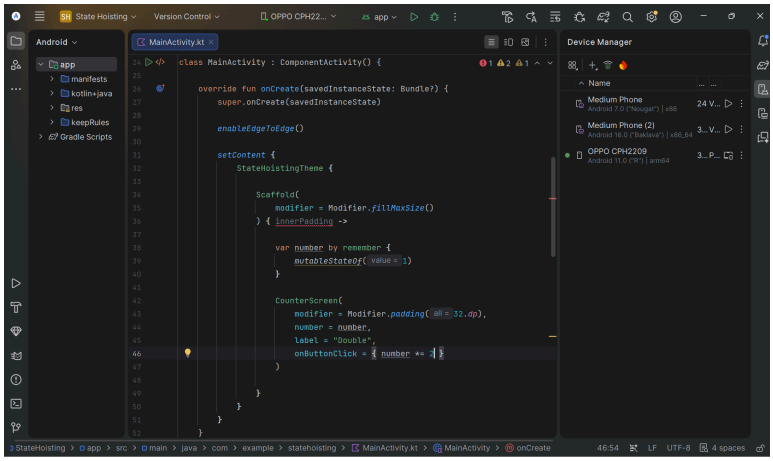
4. Hasil Program:



Jawaban:

a. Ya, program berjalan dengan baik dan tetap menampilkan perilaku yang sama seperti sebelum dilakukan state hoisting. Setiap kali tombol Increment diklik, nilai yang ditampilkan bertambah satu.
b. Proses increment dilakukan pada onButtonClick ketika CounterScreen() dipanggil di dalam onCreate(), yaitu melalui kode onButtonClick = { number++ }.
c. State number dideklarasikan di dalam onCreate() menggunakan remember dan mutableStateOf(0). Nilai label "Increment" diberikan melalui parameter label saat CounterScreen() dipanggil, sedangkan keduanya digunakan oleh CounterScreen() untuk menampilkan Text dan Button.

5. Perubahan kode program a, b, dan c:
Label tombol diubah menjadi "Double", nilai awal number diubah dari 0 menjadi 1, dan event onButtonClick diubah dari number++ menjadi number *= 2. Perubahan tersebut dilakukan pada bagian pemanggilan CounterScreen() di dalam onCreate().

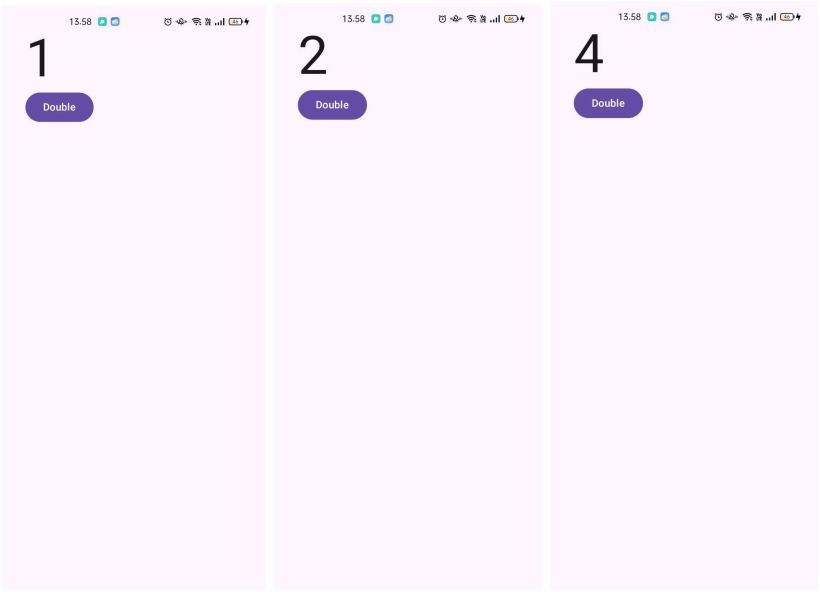


d. Kode program setelah perubahan:

```
var number by remember {
    mutableStateOf(1)
}

CounterScreen(
    modifier = Modifier.padding(32.dp),
    number = number,
    label = "Double",
    onButtonClick = { number *= 2 }
)
```

6. Hasil Program:
   a. Sebelum: Sesudah:



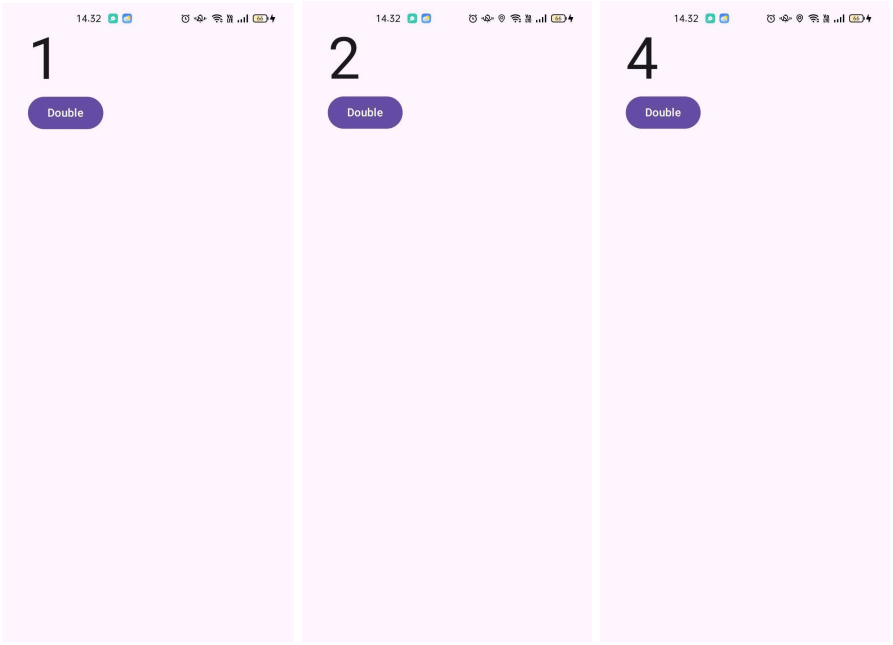
b. Perubahan pada percobaan tahap 7 dan tahap 8 dilakukan pada bagian pemanggilan CounterScreen() di dalam fungsi onCreate(). Pada tahap sebelumnya digunakan nilai awal 0, label "Increment", dan onButtonClick = { number++ }. Kemudian pada tahap berikutnya diubah menjadi nilai awal 1, label "Double", dan onButtonClick = { number *= 2 }.
c. Tidak ada perubahan yang dilakukan pada fungsi CounterScreen() itu sendiri. Struktur dan isi CounterScreen() tetap sama karena state number, label, dan event onButtonClick sudah diterima melalui parameter. Perubahan hanya dilakukan pada nilai dan logika yang diberikan saat CounterScreen() dipanggil di dalam onCreate().

7. Manfaat state hoisting pada percobaan ini adalah memisahkan pengelolaan state dan logika dari tampilan UI, sehingga perubahan kebutuhan fungsional dapat dilakukan dengan lebih mudah tanpa harus mengubah komponen CounterScreen(). Contohnya, logika dari number++ dapat diubah menjadi number *= 2, nilai awal dapat diubah, dan label tombol dapat diganti hanya melalui pemanggilan CounterScreen() di onCreate(). Sehingga, CounterScreen() menjadi lebih reusable, lebih mudah diatur, dan memiliki tanggung jawab yang lebih jelas sebagai komponen yang menampilkan UI berdasarkan data dan event yang diberikan dari luar.

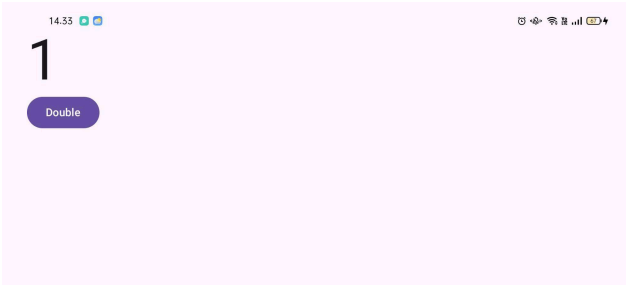
Perubahan Konfigurasi

Aktivitas Percobaan:

1. Tampilan pada device:



2. Landscape
    a. Tampilan pada device:

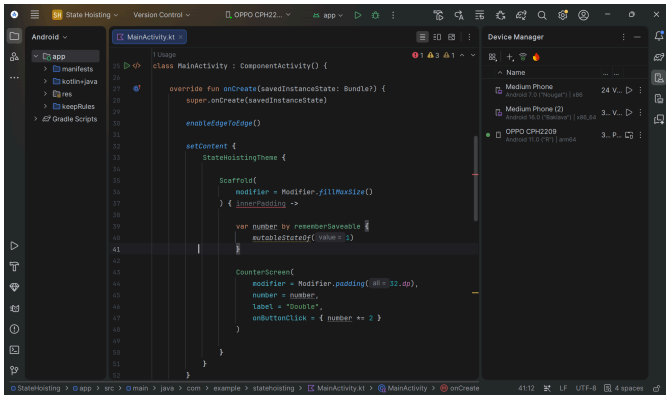






    b. Setelah perangkat diputar dari portrait menjadi landscape, nilai number kembali menjadi 1. Hal ini terjadi karena perubahan orientasi merupakan perubahan konfigurasi yang menyebabkan Activity dibuat ulang. State yang disimpan menggunakan remember tidak dapat dipertahankan ketika Activity mengalami recreation, sehingga number kembali ke nilai awal yang ditentukan pada mutableStateOf(1).

Perubahan:



Tampilan pada device:





3. Landscape:
    a. Setelah menggunakan rememberSaveable, nilai yang ditampilkan pada posisi landscape tetap sama dengan nilai terakhir sebelum perangkat diputar. Sebelum rotasi nilai terakhir adalah 8, setelah diputar ke landscape nilainya tetap 8.



    b. Angka yang ditampilkan tetap 8, hal ini karena rememberSaveable dapat menyimpan nilai state melalui mekanisme saved instance state, sehingga nilai tersebut dapat dipulihkan ketika Activity dibuat kembali akibat perubahan konfigurasi seperti rotasi layar.
    c. rememberSaveable berfungsi untuk mempertahankan dan memulihkan nilai state ketika terjadi recreation pada Activity, misalnya akibat perubahan orientasi layar. Sementara itu, remember hanya mempertahankan state selama objek composition masih berlangsung dan tidak dirancang untuk memulihkan state setelah Activity dibuat ulang. Oleh karena itu, rememberSaveable cocok digunakan untuk menyimpan data UI sederhana yang perlu tetap tersedia setelah perubahan konfigurasi.

4. Salah satu contoh penggunaannya adalah pada form pendaftaran atau pengisian data pengguna. Misalnya, pengguna sudah mengisi nama, alamat, atau pilihan pada checkbox, kemudian perangkat diputar sehingga terjadi perubahan orientasi layar. Dengan menggunakan rememberSaveable, data yang telah dimasukkan dapat dipulihkan sehingga pengguna tidak perlu mengisi ulang form dari awal. Manfaatnya adalah menjaga data sementara pada UI tetap tersedia setelah perubahan konfigurasi dan memberikan pengalaman penggunaan aplikasi yang lebih nyaman.

LaunchedEffect

Pra-Percobaan 1

Aktivitas Percobaan:

1. Menjalankan program:
    a. Pada saat pertama kali program dijalankan tanpa mengklik button "Double", layar menampilkan angka 1 dan SnackBar menampilkan pesan "Number 1 is shown!". Hasil pada device:



    b. Pesan tersebut ditampilkan karena LaunchedEffect(number) dijalankan saat aplikasi pertama kali masuk ke dalam composition. Nilai awal number adalah 1, sehingga effect langsung berjalan meskipun belum ada interaksi dari pengguna. Di dalam effect tersebut terdapat snackbarHostState.showSnackbar(), yang berfungsi untuk menampilkan pesan melalui komponen SnackBar yang sudah

terhubung dengan SnackbarHost. showSnackbar() menyebabkan pesan "Number 1 is shown!" benar-benar ditampilkan pada SnackBar.

c. Pesan pada SnackBar dipicu oleh LaunchedEffect(number). Bagian kodenya:

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar(
        "Number $number is shown!"
    )
}
```

number menjadi key pada LaunchedEffect. Saat pertama kali aplikasi dijalankan, number bernilai 1 sehingga effect dijalankan, dan akan dijalankan kembali setiap kali nilai number berubah.

2. Hasil Program:
   a. Setelah button "Double" diklik satu kali, nilai angka berubah dari 1 menjadi 2. Setelah itu, SnackBar menampilkan pesan "Number 2 is shown!".



   b. Hal tersebut terjadi karena button menjalankan:

```
onButtonClick = { number *= 2 }
```

Kode tersebut mengubah nilai number dari 1 menjadi 2. Perubahan state menyebabkan recomposition. Karena number merupakan key dari LaunchedEffect(number), perubahan nilai tersebut membuat effect dijalankan kembali.

Selanjutnya, snackbarHostState.showSnackbar("Number $number is shown!") dipanggil dengan nilai number yang baru, yaitu 2. SnackbarHost yang terdapat pada Scaffold kemudian menampilkan pesan "Number 2 is shown!" pada layar.

Pra-Percobaan 2

Aktivitas Percobaan:

1. Menjalankan program:
    a. Pada saat pertama kali program dijalankan tanpa mengklik button "Double", layar menampilkan angka 1. SnackBar menampilkan pesan "Number 1 is shown!".



    b. Setelah menunggu selama dua detik, angka pada layar berubah menjadi 799. SnackBar kemudian menampilkan pesan "Number 799 is shown!".



c. Perubahan angka dipicu oleh LaunchedEffect(Unit) yang menjalankan Repo.getData():

```
LaunchedEffect(Unit) {
    number = Repo.getData()
}
```

Fungsi Repo.getData() menjalankan delay(2000) selama dua detik, kemudian menghasilkan angka acak melalui Random.nextInt(100, 1000). Pada percobaan ini, angka yang dihasilkan adalah 799, lalu nilai tersebut disimpan ke dalam number.

Perubahan nilai number kemudian memicu LaunchedEffect(number):

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar(
        "Number $number is shown!"
    )
}
```

Karena nilai number berubah dari 1 menjadi 799, SnackBar dijalankan kembali dan menampilkan pesan "Number 799 is shown!".

2. Dari kedua percobaan
    a. Perbedaannya terletak pada key yang digunakan sebagai pemicu.
        - LaunchedEffect(number) menggunakan state number sebagai key, sehingga efek akan dijalankan kembali ketika nilai number berubah.
        - Sementara itu, LaunchedEffect(Unit) menggunakan Unit sebagai key yang nilainya tetap, sehingga efek hanya dijalankan saat pertama kali masuk ke dalam composition dan tidak dijalankan kembali hanya karena nilai number berubah.
    b. LaunchedEffect(number) dijalankan saat pertama kali masuk ke dalam composition dan akan dijalankan kembali ketika nilai number berubah. Pada percobaan ini, perubahan number dapat terjadi karena button "Double" diklik atau karena Repo.getData() memberikan nilai baru.

    Sedangkan LaunchedEffect(Unit) dijalankan saat pertama kali masuk ke dalam composition. Operasi Repo.getData() kemudian berjalan selama sekitar dua detik, lalu hasilnya diberikan ke number. Perubahan number tersebut tidak membuat LaunchedEffect(Unit) dijalankan kembali karena key-nya tetap Unit.